

ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG
NHẬP MÔN TRÍ TUỆ NHÂN TẠO - IT3160



Báo cáo Bài tập lớn

Đề tài: Xây dựng phần mềm định tuyến giao thông tại TP.HCM

Nhóm: 3 - Mã lớp: 168452

Sinh viên thực hiện:

Hà Tiến Khiêm (202416246)

Bùi Minh Cường (202416146)

Trần Văn Lợi (202416266)

Nguyễn Tùng Lâm (202416258)

Giảng viên hướng dẫn:

PGS.TS Lê Thanh Hương

Tháng 5 năm 2026

Tóm tắt nội dung

Bài toán tìm đường đi ngắn nhất là một nền tảng quan trọng trong lĩnh vực Trí tuệ Nhân tạo và các hệ thống giao thông thông minh. Tuy nhiên, tại các siêu đô thị như Thành phố Hồ Chí Minh, mạng lưới giao thông luôn ở trạng thái động; chi phí di chuyển (thời gian và quãng đường) biến thiên liên tục theo mật độ phương tiện và tình trạng ùn tắc. Để giải quyết bài toán định tuyến trong điều kiện thực tế, nghiên cứu này tiến hành xây dựng một đồ thị giao thông đa khung giờ, tích hợp dữ liệu không gian tĩnh từ OpenStreetMap và dữ liệu mức độ phục vụ (LOS) được rời rạc hóa thành 48 khung giờ trong ngày.

Mục tiêu cốt lõi của đề tài được triển khai qua ba trọng tâm chính:

- **Thứ nhất**, đề xuất và cài đặt thuật toán **Dynamic Weighted A^* (DWA^*)**. Thay vì sử dụng trọng số tĩnh, DWA^* được tích hợp một hàm heuristic động dựa trên các hàm toán học liên tục để đánh giá trọng số.
- **Thứ hai**, thực hiện đo lường và **so sánh toàn diện hiệu năng** của DWA^* với các thuật toán tìm kiếm kinh điển (Dijkstra và A^* truyền thống). Các kết quả thực nghiệm trên bộ dữ liệu quy mô lớn cho thấy DWA^* vượt trội trong việc giảm thiểu số lượng đỉnh cần duyệt và tối ưu hóa thời gian thực thi, trong khi vẫn duy trì được chất lượng đường đi ở mức gần tối ưu.
- **Thứ ba**, xây dựng thành công một **hệ thống Web GIS kiến trúc Client-Server độc lập**. Giao diện Front-end được tối ưu hóa trải nghiệm người dùng, sử dụng ngôn ngữ JavaScript thuần và thư viện bản đồ tương tác **Leaflet**.

Đề tài không chỉ khẳng định tính hiệu quả của thuật toán DWA^* trên mạng lưới đồ thị thực tế mà còn cung cấp một công cụ phần mềm hoàn chỉnh, làm tiền đề cho các ứng dụng điều hướng đô thị thông minh trong tương lai.

Link github: <https://github.com/tienkhiemsoict/astar-traffic-hcmc-routing>

Mục lục

1	Phân công nhiệm vụ	1
2	Giới thiệu bài toán	1
2.1	Phát biểu bài toán	1
2.2	Mục đích và Phạm vi nghiên cứu	2
3	Tổng quan và xử lý dữ liệu	3
3.1	Tổng quan các nguồn dữ liệu	3
3.2	Quy trình tiền xử lý và tích hợp dữ liệu	3
3.3	Cấu trúc file dữ liệu đầu ra	5
3.3.1	File nodes.csv	5
3.3.2	File edges.csv	5
4	Cơ sở lí thuyết	6
4.1	Thuật toán tìm đường đi ngắn nhất	6
4.1.1	Thuật toán Dijkstra:	6
4.1.2	Thuật toán A^*	6
4.1.3	Thuật toán DWA^*	7
4.1.4	Trực quan so sánh ba thuật toán	9
4.2	Thuật toán tìm lân cận KDTree	10
5	Thực nghiệm và Đánh giá	10
5.1	Thiết lập thực nghiệm:	10
5.2	Kết quả so sánh:	11
5.3	Đánh giá Điểm mạnh / Điểm yếu:	12
6	Cài đặt hệ thống và Khó khăn gặp phải	12
6.1	Các chức năng chính của hệ thống:	12
6.2	Khai báo các thư viện	13
6.3	Khó khăn và cách thức vượt qua	13

7 Kết luận	14
7.1 Tổng quan	14
7.2 Hướng phát triển tương lai	15
A Tài liệu và Công cụ tham khảo	17
B Mã giả các thuật toán	18

1 Phân công nhiệm vụ

Họ và tên	MSSV	Nhiệm vụ	Đánh giá
Hà Tiến Khiêm	202416246	- Xử lý và tích hợp dữ liệu không gian. - Xây dựng công thức thuật toán. - Viết báo cáo.	25%
Bùi Minh Cường	202416146	- Xây dựng công thức thuật toán. - Chạy thực nghiệm so sánh các thuật toán khác. - Phân tích và đánh giá kết quả.	25%
Trần Văn Lợi	202416266	- Xây dựng giao diện hiển thị bản đồ trực quan. - Xử lý các thao tác tương tác của người dùng. - Viết báo cáo.	25%
Nguyễn Tùng Lâm	202416258	- Tối ưu thiết kế trải nghiệm người dùng - Kiểm thử phần mềm phát hiện lỗi - Làm slide	25%

Bảng 1: Bảng phân công nhiệm vụ

2 Giới thiệu bài toán

2.1 Phát biểu bài toán

Trong lĩnh vực Khoa học Máy tính và Trí tuệ Nhân tạo, bài toán tìm đường đi ngắn nhất (*Shortest Path Problem*) trên mạng lưới đồ thị là một trong những bài toán tối ưu hóa kinh điển. Ở dạng cơ bản nhất, mạng lưới giao thông được mô hình hóa thành một đồ thị có hướng hoặc vô hướng $G = (V, E)$, trong đó V là tập hợp các đỉnh (các điểm giao cắt, ngã tư) và E là tập hợp các cạnh (các đoạn đường nối liền các đỉnh), kèm theo trọng số cố định biểu diễn khoảng cách vật lý.

Tuy nhiên, đối với môi trường giao thông thực tế tại một siêu đô thị như Thành phố Hồ Chí Minh, mô hình đồ thị tĩnh bộc lộ nhiều hạn chế. Giao thông là một hệ thống động; tốc độ và thời gian di chuyển trên một con đường không bao giờ đứng yên mà biến thiên liên tục theo từng thời điểm trong ngày. Một đoạn đường có thể thông thoáng vào lúc 10:00 sáng nhưng lại rơi vào tình trạng kẹt xe nghiêm trọng vào lúc 17:30 chiều.

Vì vậy, bài toán đặt ra trong đề tài này là **Tìm đường đi tối ưu trên đồ thị giao thông có trọng số thay đổi theo thời gian**. Cụ thể, yếu tố thời gian được rời rạc hóa thành 48 khung giờ (*time slots*) trong một ngày, mỗi khung giờ kéo dài 30 phút. Trọng số của mỗi cạnh lúc này không chỉ đơn thuần là độ dài hình học (*length*) mà là một hàm phụ thuộc vào mức độ ùn tắc (*Level of*

Service - LOS) tại chính khung giờ người dùng yêu cầu. Hệ thống cần tính toán để đưa ra lộ trình có tổng chi phí (thời gian di chuyển thực tế) nhỏ nhất từ điểm xuất phát đến điểm đích.

2.2 Mục đích và Phạm vi nghiên cứu

Mục đích nghiên cứu của đề tài tập trung vào việc giải quyết bài toán tối ưu hóa định tuyến động trong môi trường đô thị thông qua các nhiệm vụ cụ thể sau:

- **Nghiên cứu và đánh giá nền tảng:** Phân tích sâu cơ chế hoạt động, ưu điểm và hạn chế của các thuật toán định tuyến kinh điển trên đồ thị như Dijkstra và A^* truyền thống trong môi trường giao thông biến thiên.
- **Đề xuất giải pháp thuật toán cải tiến:** Cài đặt và ứng dụng biến thể Dynamic Weighted A^* (DWA*) với hàm heuristic được đánh trọng số động dựa trên khoảng cách tương đối đến đích. Giải pháp này hướng tới việc cân bằng tối ưu giữa chất lượng lời giải (tính chính xác của tuyến đường) và hiệu năng tính toán (giảm thiểu số lượng đỉnh cần duyệt, tối ưu hóa thời gian thực thi).
- **Xây dựng hệ thống thử nghiệm thực tế:** Phát triển một ứng dụng Web GIS theo kiến trúc Client-Server bằng JavaScript thuần và thư viện bản đồ tương tác Leaflet.js, so sánh trực quan hiệu năng của ba thuật toán (Dijkstra, A^* , DWA*) ngay trên giao diện bản đồ.

Phạm vi nghiên cứu của đề tài được giới hạn và xác định rõ ràng trên ba phương diện:

- **Về không gian:** Giới hạn vùng thực nghiệm trên mạng lưới giao thông đường bộ khu vực nội thành Thành phố Hồ Chí Minh. Các dữ liệu về trục đường chính và các tuyến đường nhánh được trích xuất cụ thể theo giới hạn tọa độ (Bounding Box) thông qua thư viện OSMnx.
- **Về dữ liệu:** Dữ liệu cấu trúc hình học và liên thông (Topology) của đồ thị được khai thác từ nền tảng OpenStreetMap. Dữ liệu trạng thái giao thông động được tích hợp dựa trên chỉ số mức độ phục vụ (LOS) được rời rạc hóa tương ứng với 48 khung giờ trong ngày.
- **Về phương pháp:** Tập trung tối ưu hóa cấu trúc dữ liệu không gian (sử dụng KD Tree để bắt điểm) và cấu trúc hàng đợi ưu tiên nhằm tăng tốc độ xử lý thuật toán trên đồ thị lớn có hàng chục ngàn đỉnh.

3 Tổng quan và xử lý dữ liệu

3.1 Tổng quan các nguồn dữ liệu

Hệ thống sử dụng hai nguồn dữ liệu chính để thiết lập mạng lưới giao thông động:

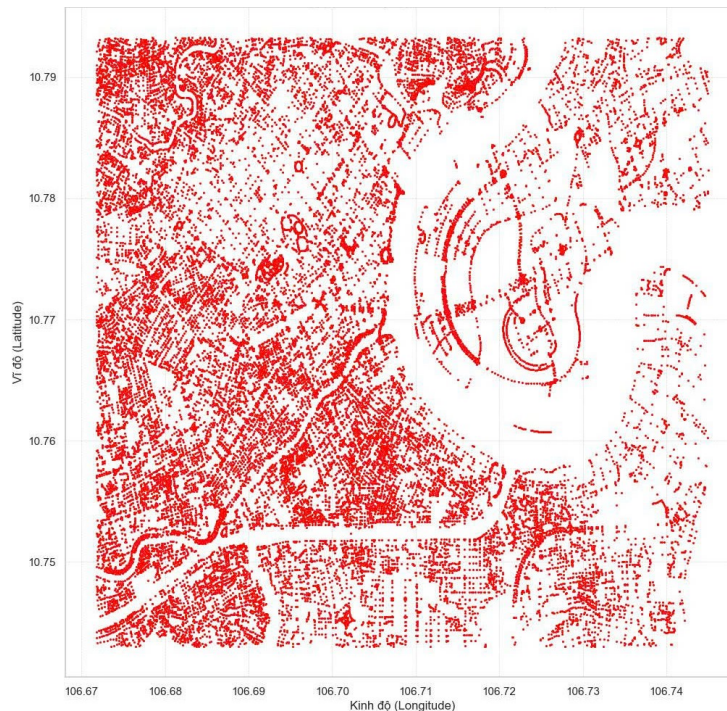
- **Dữ liệu không gian:** Được trích xuất từ OpenStreetMap (OSM) qua thư viện OSMnx, cung cấp cấu trúc topology tĩnh của mạng lưới giao thông nội thành TP.HCM.
- **Dữ liệu giao thông (Traffic Data):** Bộ dữ liệu *train.csv* từ Kaggle (Traffic Flow data in Ho Chi Minh City của Thanh Nguyen), chứa thông tin lịch sử về tình trạng ùn tắc theo mã *street_id* qua các khung giờ trong ngày.

3.2 Quy trình tiền xử lý và tích hợp dữ liệu

Quy trình tích hợp được thực hiện qua 4 bước:

- **Bước 1: Trích xuất đặc trưng không gian**

Dữ liệu thô .osm được tải về và lọc bỏ các thông tin dư thừa, chỉ trích xuất tọa độ của các đỉnh (giao lộ) và các cạnh (đoạn đường) nối giữa chúng để lưu vào mảng dữ liệu đỉnh và cạnh.



Hình 1: Hình thái không gian mạng lưới giao thông TP.HCM

• Bước 2: Quy đổi và tính toán chỉ số ùn tắc (LOS)

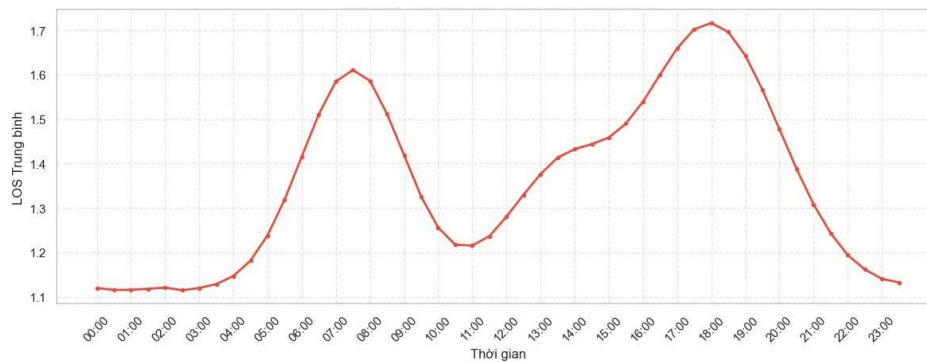
Dữ liệu train.csv chứa các nhãn phân loại kẹt xe từ A đến F được quy đổi thành các hệ số nhân cụ thể. Với các đoạn đường (*street_id*) có nhiều bản ghi trong cùng một khung giờ, hệ thống tính giá trị trung bình (mean) để đưa ra một hệ số đại diện duy nhất.

LOS	A	B	C	D	E	F
Hệ số	1.0	1.24	1.48	1.72	1.96	2.2

Bảng 2: Bảng quy đổi LOS sang trọng số

• Bước 3: Bổ sung dữ liệu thiếu

Trong thực tế, nhiều đoạn đường trên OSM không có dữ liệu đo lường trong tập Kaggle. Hệ thống thực hiện điền khuyết các giá trị thiếu này theo phân phối thống kê của từng khung giờ (ví dụ: lấy trung bình mức độ kẹt xe của toàn thành phố tại khung giờ đó) để đảm bảo tính toàn vẹn cho đồ thị.



Hình 2: Biểu đồ LOS trung bình của TP.HCM theo 48 khung giờ

• Bước 4: Tính toán trọng số cạnh tổng hợp

Sau khi đồng bộ được ID đường của OSM và Kaggle, trọng số thực tế của từng cạnh được tính toán theo công thức:

$$W_i = L \times \text{LOS}_i + P \quad (1)$$

Trong đó:

- W_i : Trọng số chi phí di chuyển tại khung giờ thứ i .
- L : Chiều dài vật lý của đoạn đường (length).

- LOS_i : Hệ số kẹt xe tại khung giờ thứ i .
- P : Hằng số phạt, chỉ được cộng thêm nếu đoạn đường thuộc loại ngõ/hẻm nhỏ nhằm hạn chế thuật toán điều hướng xe lớn đi vào.

3.3 Cấu trúc file dữ liệu đầu ra

3.3.1 File nodes.csv

- osmid: ID định danh duy nhất của đỉnh trên hệ thống OpenStreetMap.
- x: Tọa độ Kinh độ.
- y: Tọa độ Vĩ độ.

osmid	y	x
11393695758	10.7861099	106.700575
6627521383	10.7768189	106.7226041
5740298577	10.7694268	106.7252918
13513813103	10.7884087	106.6995314
13513813129	10.7870312	106.6982885
13513813132	10.7873842	106.6986084
13513813137	10.7879004	106.6991

Hình 3: nodes.csv

3.3.2 File edges.csv

- u: ID của đỉnh xuất phát.
- v: ID của đỉnh đích.
- osmid: Street ID.
- highway: Loại đường (Ví dụ: primary, secondary, residential...).
- length: Chiều dài vật lý của đoạn đường (tính bằng mét).
- geometry: Chuỗi dữ liệu hình học thực tế lưu trữ các điểm uốn cong của con đường.
- los: Mảng gồm 48 phần tử số thực đại diện cho hệ số kẹt xe tương ứng với 48 khung giờ trong ngày (mỗi khung 30 phút).

u	v	osmid	highway	length	geometry	los	weight
11393695758	13513813140	308848875	residential	36.22761	LINESTRING(106[1.0, 1.0, 1	[36.228, 3	
11393695758	6110770486	308848875	residential	101.498	LINESTRING(106[1.0, 1.0, 1	[101.498, :	
11393695758	2037831446	1228588481	service	56.17878	LINESTRING(106[1.0, 1.0, 1	[56.179, 5	
6627521383	6627521384	705425908	secondary	32.77072	LINESTRING(106[1.0, 1.0, 1	[32.771, 3:	
5740298577	13449656133	366105856	residential	20.01808	LINESTRING(106[1.15, 1.15	[23.021, 2:	
5740298577	12945991156	366105856	residential	21.23256	LINESTRING(106[1.15, 1.15	[24.417, 2:	
5740298577	5744766812	605145252	residential	42.61217	LINESTRING(106[1.0, 1.0, 1	[42.612, 4:	
13513813103	411918911	1185261855	secondary	65.43588	LINESTRING(106[1.0, 1.0, 1	[65.436, 6:	
13513813137	3141402319	308848876	service	40.88607	LINESTRING(106[1.0, 1.0, 1	[40.886, 4:	
13513813137	3141402318	308848876	service	4.129906	LINESTRING(106[1.15, 1.15	[4.749, 4.7	
366477968	5778163356	286278359	service	42.67887	LINESTRING(106[1.0, 1.0, 1	[42.679, 4:	
6573654683	5740394720	700017574	residential	111.626	LINESTRING(106[1.0, 1.0, 1	[111.626, :	

Hình 4: edges.csv

4 Cơ sở lí thuyết

4.1 Thuật toán tìm đường đi ngắn nhất

Trong bài toán tìm đường đi ngắn nhất Dijkstra và A* là hai thuật toán nền tảng:

4.1.1 Thuật toán Dijkstra:

Là một dạng tìm kiếm theo chiều rộng (Breadth-First Search) có trọng số. Thuật toán đảm bảo tìm được đường đi ngắn nhất bằng cách mở rộng tất cả các nút xung quanh cho đến khi chạm tới đích. Tuy nhiên, điểm yếu lớn nhất là nó mở rộng vùng tìm kiếm theo mọi hướng, dẫn đến việc duyệt qua quá nhiều nút không cần thiết, gây tốn tài nguyên và thời gian.

4.1.2 Thuật toán A*

Cải tiến từ thuật toán Dijkstra bằng cách sử dụng hàm heuristic $h(n)$ để lựa chọn (định hướng) tìm kiếm. Thuật toán sử dụng hàm đánh giá:

$$f(n) = g(n) + h(n)$$

. Trong đó:

- $g(n)$: chi phí từ nút gốc đến nút hiện tại n
- $h(n)$: chi phí ước lượng từ nút hiện tại n đến đích

Trong các bài toán tìm đường đi trên bản đồ, hàm heuristic phổ biến gồm khoảng cách Euclid, khoảng cách Manhattan và khoảng cách Haversine.

Mặc dù nhanh hơn Dijkstra, nhưng trong môi trường phức tạp hoặc bản đồ lớn, A^* vẫn phải duyệt nhiều nút trước khi tìm thấy mục tiêu, đặc biệt khi hàm $h(n)$ không đủ mạnh để "kéo" thuật toán về phía đích một cách quyết liệt.

4.1.3 Thuật toán DWA^*

Xuất phát từ thuật toán A^* truyền thống, để tối ưu hóa tốc độ tìm kiếm chúng em đề xuất sử dụng thuật toán DWA^* (Dynamic Weighted A-star). Thay vì giữ trọng số hằng số cho hàm heuristic $h(n)$, DWA^* điều chỉnh tầm ảnh hưởng của $h(n)$ thông qua 1 tham số động w . Thuật toán này sử dụng hàm đánh giá:

$$f(n) = g(n) + w.h(n)$$

Trong đó :

- $g(n)$: chi phí từ nút gốc đến nút hiện tại n
- $h(n)$: chi phí ước lượng từ nút hiện tại n đến đích

Chi tiết về các hàm chi phí và tham số động:

- Hàm $g(n)$

Hàm chi phí đường đi $g(n)$ biểu thị khoảng cách chính xác, đã biết từ nút bắt đầu ban đầu đến vị trí hiện tại trong quá trình tìm kiếm. Không giống như các giá trị ước tính, chi phí này chính xác và được tính bằng cách cộng tất cả các trọng số cạnh riêng lẻ đã được đi qua dọc theo đường đi mà chúng ta đã duyệt.

Về mặt toán học, đối với một đường đi từ nút n_{start} cho đến nút $n_{current}$ ta có thể biểu diễn $g(n_{current})$ như sau:

$$g(n_k) = \sum_{i=0}^n w(n_i, n_{i+1})$$

- Hàm heuristic $h(n)$

Hàm heuristic $h(n)$ cung cấp chi phí ước tính từ nút hiện tại đến nút đích, đóng vai trò là "phỏng đoán có cơ sở" của thuật toán về đường đi còn lại.

Về mặt toán học, đối với bất kỳ nút n nào, ước tính heuristic phải thỏa mãn điều kiện $h(n) \leq h^*(n)$, trong đó $h^*(n)$ là chi phí thực tế để đạt được mục tiêu, khiến nó được chấp nhận vì không bao giờ đánh giá quá cao chi phí thực sự.

Trong thuật toán DWA^* , chúng em xác định hàm $h(n)$ bằng công thức Euclidean - Một hàm được sử dụng để tính toán khoảng cách theo đường chim bay giữa hai tọa độ địa lý. Đây là một hàm ước lượng (heuristic) đáng tin cậy vì khoảng cách đường chim bay luôn nhỏ hơn hoặc bằng khoảng cách thực tế trên đường bộ.

- Tham số động w

Tham số động w nhằm mục đích cân bằng giữa tốc độ (thời gian tìm đường) và tính tối ưu.

- Ở giai đoạn xa đích: Chúng em muốn thuật toán chạy cực nhanh, “nhắm thẳng” về phía đích mà không cần quan tâm quá chi tiết đến các ngách nhỏ. Khi này tham số w nên lớn để ưu tiên hàm heuristic $h(n)$ “tham lam” hơn.
- Ở giai đoạn gần đích: Khi đã gần mục tiêu, nếu tham số w còn lớn, thuật toán tham lam dẫn đến lệch hẳn khỏi đường đi tối ưu hoặc không tìm được đường đến đích. Lúc này tham số w nên giảm dần về 1 để thuật toán DWA^* hoạt động như thuật toán A^* truyền thống đảm bảo cập bến an toàn và tối ưu.

Từ đó có thể thấy tham số động w có thể có nhiều cách thiết lập. Qua nghiên cứu và thực nghiệm, nhóm chúng em đã đề xuất ra 1 phương pháp để thiết lập tham số động w . Trong đó w được tính bởi công thức:

$$w(n) = 1 + C \cdot \frac{h(cur \rightarrow goal)}{h(start \rightarrow goal)}$$

Trong đó $h(cur \rightarrow goal)$ là khoảng cách Euclidean từ điểm hiện tại cho đến điểm đích, $h(start \rightarrow goal)$ là khoảng cách Euclidean từ điểm đầu cho đến điểm đích.

- Hàm chi phí tổng $f(n)$

Tổng chi phí ước tính $f(n)$ là nền tảng của quá trình ra quyết định của thuật toán A^* , kết hợp cả chi phí đường đi thực tế và ước tính kinh nghiệm để đánh giá tiềm năng của mỗi

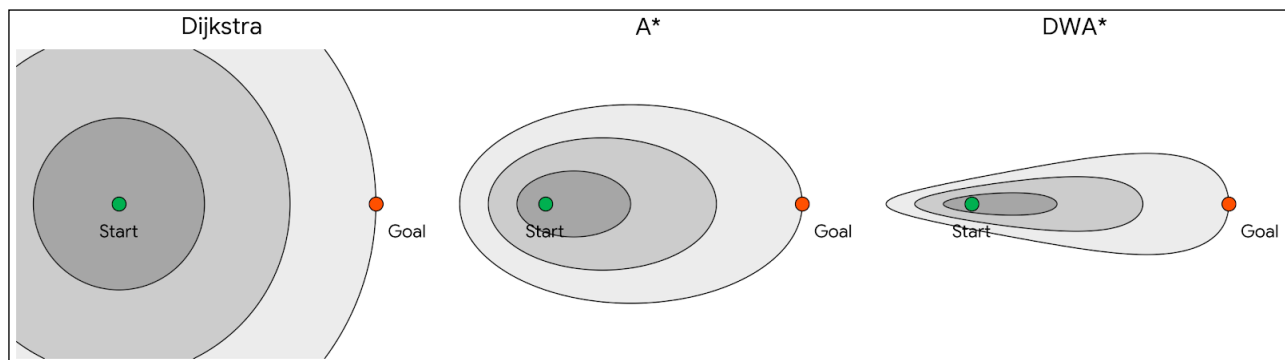
nút. Đối với bất kỳ nút n nào, chi phí này được tính như sau:

$$f(n) = g(n) + [1 + C \cdot \frac{h(n)}{h(start \rightarrow goal)}] \cdot h(n)$$

Ý nghĩa:

- Khi ở gần điểm xuất phát: $h(n)$ xấp xỉ $h(start \rightarrow goal)$, điều này dẫn đến tham số $w(n)$ lớn $\Rightarrow$ Thuật toán tham lam hơn, hướng mục tiêu thẳng về phía đích và giảm số nodes cần duyệt
- Khi tiến gần tới đích: $h(n) \rightarrow 0$, điều này dẫn đến tham số $w(n)$ xấp xỉ 1 $\Rightarrow$ Thuật toán trở về A^* truyền thống, đảm bảo tính chính xác và tối ưu khi tìm đường đến đích.
- Trong project của nhóm chúng em, với bộ dữ liệu là tuyến đường giao thông Thành phố Hồ Chí Minh và tình trạng giao thông qua các khung giờ. Qua thực nghiệm tìm tuyến đường ngắn nhất giữa 2 điểm bất kỳ trên bản đồ, chúng em quyết định lựa chọn $C = 0.5$ vì con số này đảm bảo thuật toán gần tối ưu (Near optimal) mà ít tiêu tốn bộ nhớ hơn (do số lượng nodes duyệt ít hơn) và tốc độ tìm kiếm nhanh vượt trội so với thuật toán Dijkstra, A^* truyền thống - phù hợp với bài toán tìm đường trong dữ liệu địa lý thực tế. (Về quá trình thực nghiệm chứng minh sẽ trình bày chi tiết ở Chương 5).

4.1.4 Trực quan so sánh ba thuật toán



Hình 5: Trực quan so sánh ba thuật toán

Chú thích:

- **Các đường đồng mức:** Nối các đỉnh có cùng mức chi phí đánh giá tại một thời điểm.

- **Phân bố màu sắc (đậm đến nhạt):** Thể hiện các mức chi phí tăng dần khi thuật toán liên tục mở rộng tìm kiếm lúc chưa chạm đến đích.
- **Tương quan diện tích:** Diện tích của các vùng màu càng bé đồng nghĩa với số lượng node phải duyệt càng ít, minh chứng cho việc thuật toán ép không gian tìm kiếm hiệu quả.

4.2 Thuật toán tìm lân cận KDTree

KD-Tree (K-Dimensional Tree) là cấu trúc cây nhị phân phân vùng không gian đa chiều, dùng để tăng tốc độ tìm kiếm. Trong các ứng dụng bản đồ, đây là công cụ cốt lõi để truy vấn vị trí lân cận (Nearest Neighbor)

5 Thực nghiệm và Đánh giá

5.1 Thiết lập thực nghiệm:

Quy trình được tiến hành tuần tự theo 4 bước sau:

- **Bước 1:** Thiết lập môi trường phần cứng, phần mềm (ngôn ngữ Python, Microsoft Excel) và nạp các tệp dữ liệu cấu trúc đồ thị (`nodes.csv`, `edges.csv`).
- **Bước 2:** Lựa chọn ngẫu nhiên các cặp đỉnh (start - goal) từ tệp `nodes.csv` để chạy các thuật toán định tuyến. Nhóm tiến hành chọn ra các khung giờ giao thông tiêu biểu (giờ cao điểm, giờ thông thoáng) và thực hiện lặp lại khoảng 4000 lượt truy vấn tìm đường cho mỗi khung giờ.
- **Bước 3:** Phân loại các kịch bản tìm đường thành 3 nhóm dựa trên chiều dài quãng đường: khoảng cách ngắn (< 5 km), trung bình ($5 - 10$ km) và dài (> 10 km). Từ đó, tính toán các tham số trung bình cho mỗi nhóm bao gồm: chiều dài lộ trình, số lượng đỉnh đã duyệt (visited nodes), và thời gian thực thi tính toán.
- **Bước 4:** Tổng hợp bộ dữ liệu kết quả xuất ra Microsoft Excel để lập bảng thống kê, qua đó so sánh và phân tích đối chiếu hiệu năng giữa thuật toán cơ sở và thuật toán đề xuất.

5.2 Kết quả so sánh:

Bảng 3: Thống kê thực nghiệm và so sánh hiệu năng các thuật toán

Thuật toán	< 5 km			5 ≤ distance < 10 km			≥ 10 km		
	Gap	Node	Time (ms)	Gap	Node	Time (ms)	Gap	Node	Time (ms)
Dijkstra	0.000%	9336	29.280	0.000%	23797	61.192	0.000%	31280	76.543
A* Origin	0.000%	2633	7.690	0.000%	9943	30.270	0.000%	19245	60.558
DWA* (c = 0.5)	1.266%	1367	4.009	0.970%	5536	17.223	0.351%	13089	42.905
DWA* (c = 1.0)	3.616%	844	2.408	3.472%	3232	9.920	2.252%	7833	25.095
DWA* (c = 1.5)	5.625%	626	1.726	5.929%	2176	6.541	4.625%	4845	15.070
DWA* (c = 2.0)	7.159%	528	1.427	7.917%	1695	5.061	7.187%	3588	11.076

*Chú thích:

- **Gap:** Độ lệch chi phí quãng đường di chuyển so với thuật toán chính xác (càng nhỏ càng tốt).
- **Node:** Số lượng đỉnh trung bình phải duyệt để tìm ra đường đi (càng nhỏ càng tốt).
- **Time:** Thời gian thực thi tính toán trung bình (càng nhỏ càng tốt).

*Trong nhóm thuật toán gần tối ưu, kết quả **tốt nhất** được **tô đậm**, kết quả tốt nhì được gạch chân.

- Với khoảng cách ngắn (< 5 km), DWA* (C = 0.5) cực kỳ hiệu quả. Nó duyệt nhanh hơn Dijkstra 7 lần, hơn A* 1,92 lần, giảm hơn 85% số node duyệt so với Dijkstra, 48% số node duyệt so với A* trong khi đường đi chỉ dài hơn khoảng 1.27%.
- Ở khoảng cách trung bình (5-10 km), DWA* (C = 0.5) vẫn giữ hiệu quả tốt: nhanh hơn Dijkstra 3.55 lần, nhanh hơn A* Origin 1.76 lần, nhưng Gap dưới 1%.
- Ở khoảng cách dài (> 10 km), lợi thế tốc độ giảm dần, nhưng chi phí đường đi lại rất tốt: chỉ hơn 0.35%. Điều này cho thấy C = 0.5 khá ổn định khi khoảng cách lớn.
- So sánh các phiên bản DWA* với C cao hơn:
 - C = 1.0 nhanh hơn C = 0.5 khoảng 1.71 lần, nhưng Gap tăng lên 2.25%
 - C = 1.5 nhanh hơn C = 0.5 khoảng 2.75 lần, nhưng Gap tăng lên 4.53%
 - C = 2.0 nhanh hơn C = 0.5 khoảng 3.65 lần, nhưng Gap tăng lên 6.56%

Kết quả thực nghiệm cho thấy DWA* với C = 0.5 đạt được sự cân bằng tốt giữa hiệu năng tính toán và chất lượng đường đi. So với Dijkstra, thuật toán này giảm trung bình khoảng 68.96% số node duyệt và tăng tốc khoảng 2.60 lần, trong khi độ lệch độ dài đường đi chỉ khoảng 0.86%. So với A* Origin, DWA* C = 0.5 vẫn nhanh hơn khoảng 1.54 lần và giảm 37.18% số node duyệt. Đặc

biệt với quãng đường càng xa thì chênh lệch độ dài đường đi so với 2 thuật toán có dấu hiệu giảm, chất lượng đường đi tốt hơn. Điều này cho thấy việc sử dụng trọng số động giúp thuật toán giảm đáng kể không gian tìm kiếm mà vẫn duy trì đường đi gần tối ưu.

Khi tăng hệ số C , các phiên bản DWA^* $C = 1.0$, $C = 1.5$ và $C = 2.0$ tiếp tục cải thiện tốc độ và giảm số node duyệt, tuy nhiên Gap tăng đáng kể. Cụ thể, DWA^* $C = 2.0$ nhanh hơn DWA^* $C = 0.5$ khoảng 3.65 lần nhưng Gap trung bình tăng từ 0.86% lên 7.42%. Do đó, $C = 0.5$ phù hợp hơn trong các bài toán yêu cầu đường đi có chất lượng cao và cân bằng giữa tốc độ và chất lượng đường đi, còn các giá trị C lớn phù hợp khi ưu tiên tốc độ xử lý hơn độ tối ưu của đường đi.

5.3 Đánh giá Điểm mạnh / Điểm yếu:

Điểm mạnh: Số đỉnh phải duyệt giảm cực mạnh, thời gian tìm đường siêu tốc.

Điểm yếu: Thuật toán không còn đảm bảo tối ưu tuyệt đối như Dijkstra/ A^* Origin. Kết quả thực nghiệm cho thấy đường đi tìm được chỉ là sub-optimal (gần tối ưu chứ không phải tốt nhất tuyệt đối). Với các đồ thị gặp nhiều vật cản (sông, hồ, đường hỏng,...), trường hợp xấu nhất có thể trở thành Dijkstra hoặc quãng đường không còn tối ưu.

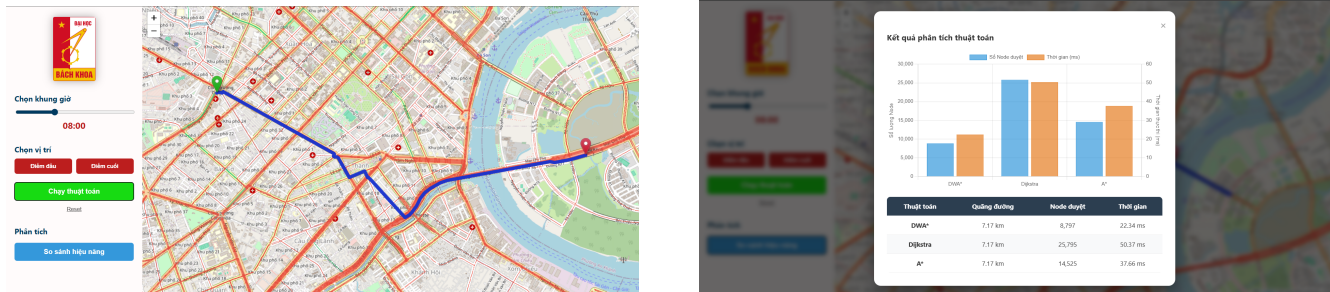
6 Cài đặt hệ thống và Khó khăn gặp phải

6.1 Các chức năng chính của hệ thống:

Hệ thống được thiết kế dưới dạng một Dashboard tương tác thời gian thực, tập trung vào trải nghiệm người dùng và tính chính xác của thuật toán:

- Tương tác bản đồ thông minh: Cho phép người dùng chọn tọa độ xuất phát và đích bằng cách click trực tiếp trên bản đồ Folium. Hệ thống tự động sử dụng cơ chế **Snap-to-Node** (hít vào điểm gần nhất) để đảm bảo tính hợp lệ cho thuật toán đồ thị.
- Tùy biến tham số đầu vào: Hỗ trợ lựa chọn khung giờ di chuyển để cập nhật dữ liệu mật độ giao thông thực tế (Traffic Density), từ đó thay đổi trọng số của các cạnh trong đồ thị.
- Phân tích và So sánh đa thuật toán: Thực thi đồng thời các thuật toán Dijkstra, A^* Origin, và DWA^* .

- Kết xuất báo cáo trực quan: Hiển thị bảng so sánh chi tiết về: Thời gian thực thi (ms), Chi phí đường đi (Cost), và Số lượng Node đã duyệt (Visited Nodes) ngay trên Sidebar. Đồng thời, vẽ lộ trình tối ưu lên bản đồ HTML động.



Hình 6: Một số chức năng của hệ thống

6.2 Khai báo các thư viện

- **FastAPI & Uvicorn:** Xây dựng, vận hành máy chủ REST API để nhận và trả kết quả.
- **Pandas & NumPy:** Xử lý dữ liệu đồ thị (CSV), lưu trữ và tính toán LOS
- **Shapely:** Xử lý hình học không gian và chuyển đổi tọa độ WKT sang GeoJSON.
- **SciPy:** Triển khai KD-Tree để tìm nhanh đỉnh đồ thị gần nhất với vị trí người dùng.
- **Pydantic:** Xác thực và ép kiểu dữ liệu đầu vào từ Frontend.
- **CORS Middleware:** Cấp quyền cho Frontend độc lập gọi API nội bộ.

6.3 Khó khăn và cách thức vượt qua

Sự khuyết thiếu dữ liệu kẹt xe theo khung giờ

- Vấn đề: Không phải toàn bộ cung đường trên mạng lưới OpenStreetMap đều có báo cáo đo lường mức độ kẹt xe trong tập dữ liệu Kaggle. Rất nhiều tuyến đường bị khuyết dữ liệu (mang giá trị NaN) ở một số mốc thời gian nhất định, hoặc hoàn toàn không có thông tin đo lường trong suốt 48 khung giờ.

- Cách thức vượt qua: Nhóm đã sử dụng kỹ thuật nội suy dữ liệu (Data Imputation). Cụ thể, các giá trị mức độ phục vụ (LOS) bị khuyết được điền bù tự động dựa trên phân phối dữ liệu trung bình theo giờ. Đối với các đoạn hầm nhỏ không có dữ liệu lịch sử, nhóm tiến hành gán hệ số LOS mặc định tương đương mức trung bình.

Khó khăn trong việc đồng bộ hóa dữ liệu tương tác:

- Vấn đề: Dữ liệu tọa độ từ cú click chuột của người dùng thường không trùng khớp với tọa độ thực tế của các nút (Nodes) trong đồ thị giao thông. Nếu sử dụng tọa độ thô, các thuật toán tìm đường của thành viên khác sẽ không thể khởi chạy hoặc cho ra kết quả sai.
- Cách thức vượt qua: Chúng em đã thiết lập một lớp trung gian xử lý sự kiện click. Ngay khi nhận được tọa độ từ bản đồ, hệ thống sẽ thực hiện truy vấn nhanh qua KD-Tree để tìm Node gần nhất và phản hồi bằng cách "hít" Marker vào đúng vị trí mặt đường. Điều này giúp chuẩn hóa dữ liệu đầu vào cho bộ phận xử lý thuật toán.

Thách thức khi thiết kế công thức hệ số trọng số động w

- Vấn đề: Việc kết hợp trực tiếp hàm heuristic $h(n)$ (vốn mang đơn vị khoảng cách hoặc thời gian) với các hằng số điều chỉnh thuần túy mà thiếu đi bước triệt tiêu đơn vị đã làm sai lệch cấu trúc của hàm tổng chi phí $f(n)$, khiến thuật toán mất khả năng định hướng chính xác.
- Cách thức vượt qua: Xây dựng công thức đưa hệ số về dạng đại lượng không thứ nguyên. Cụ thể, giá trị khoảng cách ước lượng hiện tại $h(n)$ được chia cho tổng khoảng cách ước tính của toàn bộ lộ trình D_{total} để lấy tỷ lệ phần trăm quãng đường còn lại, sau đó mới kết hợp cùng hằng số điều chỉnh C . Giải pháp này giúp công thức hệ số $w(n)$ hoàn toàn đồng nhất về mặt đơn vị.

7 Kết luận

7.1 Tổng quan

Trải qua quá trình nghiên cứu và thực nghiệm, hệ thống đã chứng minh được tính hiệu quả trong việc giải quyết bài toán tìm đường thông minh tại khu vực TP.HCM:

- Hiệu quả của thuật toán DWA* : So với các thuật toán truyền thống như Dijkstra hay A*, DWA* đã thể hiện sự vượt trội trong việc thích ứng với dữ liệu giao thông động. Bằng cách thay đổi trọng số theo các khung giờ cao điểm và thấp điểm, thuật toán không chỉ tìm ra quãng đường ngắn nhất về mặt vật lý mà còn tối ưu hóa về mặt thời gian di chuyển thực tế.
- Tối ưu hóa hệ thống: Việc áp dụng cấu trúc dữ liệu KD-Tree đã giúp hệ thống vận hành mượt mà, xử lý hàng chục nghìn Node dữ liệu mà không gây trễ.
- Giá trị thực tiễn: Ứng dụng đã cung cấp một công cụ trực quan, giúp người dùng dễ dàng so sánh hiệu năng giữa các thuật toán và hiểu rõ tác động của mật độ giao thông đến việc lựa chọn lộ trình tại các đô thị lớn như TP.HCM.

7.2 Hướng phát triển tương lai

Để hệ thống hoàn thiện hơn và có khả năng ứng dụng rộng rãi trong thực tế. Một số hướng phát triển tiêu biểu có thể kể đến như sau:

- Tích hợp Machine Learning để tối ưu hóa tham số.
 - Tự động hóa hằng số 'C': Hiện tại, hằng số 'C' trong thuật toán DWA* vẫn đang được thiết lập thủ công dựa trên kinh nghiệm. Hướng phát triển tiếp theo là xây dựng một mô hình **Machine Learning** (như Random Forest hoặc Neural Networks) để phân tích dữ liệu lịch sử giao thông.
 - Mục tiêu: Mô hình sẽ tự động dự đoán và đưa ra giá trị C tối ưu nhất cho từng đoạn đường cụ thể và từng khung giờ đặc thù (lễ tết, trời mưa, tai nạn), giúp lộ trình tìm được đạt độ chính xác cao nhất so với thực tế.
- Nâng cấp trải nghiệm tương tác người dùng (UI/UX)
 - Tương tác trực tiếp toàn diện: Phát triển sâu hơn giao diện bản đồ, cho phép người dùng không chỉ click chọn điểm đầu/cuối mà còn có thể kéo thả (Drag and Drop) các Marker để thay đổi lộ trình tức thời.
 - Hệ thống dẫn đường thời gian thực: Tích hợp GPS để định vị vị trí hiện tại của người dùng, thay thế việc chọn điểm xuất phát thủ công bằng chức năng "Vị trí của tôi".

- Đa nền tảng: Chuyển đổi giao diện Web hiện tại sang dạng Progressive Web App (PWA) hoặc ứng dụng di động để người dùng có thể dễ dàng sử dụng khi đang tham gia giao thông trên đường.
- Mở rộng quy mô dữ liệu và tính năng phụ trợ
 - Cập nhật dữ liệu Real-time: Kết nối với API của các đơn vị quản lý giao thông để lấy dữ liệu kẹt xe theo thời gian thực thay vì chỉ dựa trên các khung giờ cố định trong file CSV
 - Tính năng tránh điểm đen: Bổ sung các tùy chọn như "Tránh đoạn đường đang thi công" hoặc "Tránh vùng ngập nước" dựa trên phản hồi từ cộng đồng người dùng.

A Tài liệu và Công cụ tham khảo

- **Dữ liệu không gian OpenStreetMap (OSM) và Thư viện OSMnx**

Nền tảng OSM cung cấp dữ liệu hình học, tọa độ và cấu trúc liên thông (topology) của toàn bộ mạng lưới đường bộ nội thành TP.HCM. Thư viện Python *OSMnx* được sử dụng để trích xuất, làm sạch và chuyển đổi dữ liệu thô này thành cấu trúc đồ thị đa hướng phục vụ cho việc tính toán.

<https://osmnx.readthedocs.io/>

- **Bộ dữ liệu Traffic Flow in Ho Chi Minh City (Kaggle)**

Bộ dữ liệu lịch sử đo lường mức độ kẹt xe tại TP.HCM. Dữ liệu này được nhóm sử dụng để ánh xạ hệ số mức độ phục vụ (LOS) theo 48 khung giờ vào từng đoạn đường, làm cơ sở để tính toán trọng số động cho thuật toán DWA*.

<https://www.kaggle.com/datasets/thanhnguyen2612/traffic-flow-data-in-ho-chi-minh-city-viet-nam/>

- **Thư viện bản đồ tương tác Leaflet JS**

Leaflet là một thư viện JavaScript mã nguồn mở được nhóm sử dụng làm công cụ render chính cho giao diện Web GIS ở phía Frontend.

<https://leafletjs.com/>

- **Các gói phần mềm và thư viện lập trình**

Hệ thống kế thừa các thư viện mã nguồn mở có sẵn của Python để tối ưu hóa việc xử lý số liệu lớn.

- **Cơ sở lý thuyết thuật toán Đồ thị**

Các khái niệm nền tảng về đồ thị có trọng số, cơ chế hoạt động của thuật toán Dijkstra, thuật toán heuristic A^* được tham khảo từ bài giảng môn học do PGS.TS Lê Thanh Hương giảng dạy.

B Mã giả các thuật toán

Algorithm 1: Thuật toán Dijkstra

```

1 Dijkstra ( $N, A, n_0, GOAL$ )
2 {  $g(n_0) \leftarrow 0$ ;
3 fringe  $\leftarrow \{n_0\}$ ;
4 closed  $\leftarrow \emptyset$ ;
5 while ( $fringe \neq \emptyset$ ) do
6    $n \leftarrow GET\_FIRST(fringe)$ ;
7   closed  $\leftarrow closed \cup \{n\}$ ;
8   if ( $n \in GOAL$ ) then
9     return SOLUTION( $n$ );
10  if ( $\Gamma(n) \neq \emptyset$ ) then
11    foreach  $m \in \Gamma(n)$  do
12      if ( $m \notin closed$ ) then
13         $g_{new} \leftarrow g(n) + cost(n, m)$ ;
14        if ( $m \notin g$  or  $g_{new} < g(m)$ ) then
15           $g(m) \leftarrow g_{new}$ ;
16           $f(m) \leftarrow g(m)$ ;
17          fringe  $\leftarrow INSERT(m, f(m), fringe)$ ;
18 return ("No solution");
19 }
```

Algorithm 2: Thuật toán A^*

```

1   $A^*(N, A, n_0, GOAL)$ 
2   $\{ g(n_0) \leftarrow 0;$ 
3   $fringe \leftarrow \{n_0\};$ 
4   $closed \leftarrow \emptyset;$ 
5  while ( $fringe \neq \emptyset$ ) do
6       $n \leftarrow GET\_FIRST(fringe);$ 
7       $closed \leftarrow closed \cup \{n\};$ 
8      if ( $n \in GOAL$ ) then
9           $\quad$  return  $SOLUTION(n);$ 
10     if ( $\Gamma(n) \neq \emptyset$ ) then
11         foreach  $m \in \Gamma(n)$  do
12             if ( $m \notin closed$ ) then
13                  $g_{new} \leftarrow g(n) + cost(n, m);$ 
14                 if ( $m \notin g$  or  $g_{new} < g(m)$ ) then
15                      $g(m) \leftarrow g_{new};$ 
16                      $h(m) \leftarrow Distance(m, GOAL);$ 
17                      $f(m) \leftarrow g(m) + h(m);$ 
18                      $fringe \leftarrow INSERT(m, f(m), fringe);$ 
19 return ("No solution");
20 }
```

Algorithm 3: Thuật toán DWA*

```

1  DWA* ( $N, A, n_0, \text{GOAL}$ )
2  {  $D_{total} \leftarrow \text{Distance}(n_0, \text{GOAL})$ ;
3   $g(n_0) \leftarrow 0$ ;
4  fringe  $\leftarrow \{n_0\}$ ;
5  closed  $\leftarrow \emptyset$ ;
6  while ( $\text{fringe} \neq \emptyset$ ) do
7       $n \leftarrow \text{GET\_FIRST}(\text{fringe})$ ;
8      closed  $\leftarrow \text{closed} \cup \{n\}$ ;
9      if ( $n \in \text{GOAL}$ ) then
10         return  $\text{SOLUTION}(n)$ ;
11     if ( $\Gamma(n) \neq \emptyset$ ) then
12         foreach  $m \in \Gamma(n)$  do
13             if ( $m \notin \text{closed}$ ) then
14                  $g_{new} \leftarrow g(n) + \text{cost}(n, m)$ ;
15                 if ( $m \notin g$  or  $g_{new} < g(m)$ ) then
16                      $g(m) \leftarrow g_{new}$ ;
17                      $h(m) \leftarrow \text{Distance}(m, \text{GOAL})$ ;
18                      $w(m) \leftarrow 1.0 + 0.5 \times \left( \frac{h(m)}{D_{total}} \right)$ ;
19                      $f(m) \leftarrow g(m) + w(m) \times h(m)$ ;
20                     fringe  $\leftarrow \text{INSERT}(m, f(m), \text{fringe})$ ;
21 return ("No solution");
22 }
```

Algorithm 4: Thuật toán Tìm kiếm lân cận gần nhất trên KD-Tree

```

1 KD_NEAREST ( $n, P, depth, best$ )
2 { if ( $n = NULL$ ) then
3   | return  $best$ ;
4 if ( $Distance(n, P) < Distance(best, P)$ ) then
5   |  $best \leftarrow n$ ;
6  $axis \leftarrow depth \bmod 2$ ;
7 if ( $P[axis] < n[axis]$ ) then
8   |  $next\_node \leftarrow n.left$ ;
9   |  $oppo\_node \leftarrow n.right$ ;
10 else
11   |  $next\_node \leftarrow n.right$ ;
12   |  $oppo\_node \leftarrow n.left$ ;
13  $best \leftarrow \mathbf{KD\_NEAREST}(next\_node, P, depth + 1, best)$ ;
14 if ( $|P[axis] - n[axis]| < Distance(best, P)$ ) then
15   |  $best \leftarrow \mathbf{KD\_NEAREST}(oppo\_node, P, depth + 1, best)$ ;
16 return  $best$ ;
17 }
```
